Early Diagnostics for Sender Expressions

Document:
P3164R3
Authors:
Eric Niebler
Date:
Jan 9, 2025
Source:
Github
Issue tracking:
Github
Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++
Audience:
Library Evolution Working Group

Synopsis

This paper aims to improve the user experience of the sender framework by giving users immediate feedback about incorrect sender expressions.

A relatively minor change to how sender completion signatures are computed, and a trivial change to the sender adaptor algorithms makes it possible for the majority of sender expressions to be type-checked immediately, when the expression is constructed, rather than when it is connected to a receiver (the status quo).

Executive Summary

Below are the specific changes this paper proposes in order to improve the diagnostics emitted by sender-based codes:

1. Define a "non-dependent sender" to be one whose completions are knowable without an environment.

2. Add support for calling `get_completion_signatures` without an environment argument.

3. Change the definition of the `completion_signatures_of_t` alias template to support querying a sender's non-dependent signatures, if such exist.

4. Extend the awaitable helper concepts to support querying a type whether it is awaitable in an arbitrary coroutine (without knowing the promise type). For example, anything that implements the awaiter interface (`await_ready`, `await_suspend`, `await_resume`) is awaitable in any coroutine, and should function as a non-dependent sender.

5. Require the sender adaptor algorithms to preserve the "non-dependent sender" property wherever possible.

6. Add "Mandates:" paragraphs to the sender adaptor algorithms to require them to hard-error when passed non-dependent senders that fail type-checking.

7. Extend the eager type checking of the `let_` family of algorithms to hard-error if the user passes a lambda that does not return a sender type.

8. For any algorithm that eagerly `connect`s a sender (e.g., `sync_wait`, `split`), hard-error (*i.e.* `static_assert`) if the sender fails to type-check rather than SFINAE-ing the overload away.

9. Specify that `run_loop`'s schedule sender is non-dependent.

Revision History

R3:

- Rebase the paper on the current standard working draft.

- Remove section respecifying `transform_completion_signatures` to propagate type errors. A separate paper (P3557) addresses the issue of type errors during the computation of completion signatures.

- Specify `stopped_as_optional` to mandate that its child sender satisfies the *`single-sender`* concept, and change the *`single-sender`* concept so that it works properly with non-dependent senders.

- Add requirement to [exec.snd.general] that ensures user-defined customizations of sender algorithms produce non-dependent senders when the default implementation would.

- Specify the exposition-only *`basic-sender`* helper to support the creation of non-dependent senders. (This change includes the proposed resolution for cplusplus/sender-receiver#307.)

- Update the exposition-only *`sender-of`* concept to work with non-dependent senders (i.e. *`sender-of`*`<Sndr, int>` subsumes `sender_in<Sndr>`).

- Specify that the sender returned by calling `schedule` on `run_loop`'s scheduler is non-dependent.

R2:

- Remove the `sender_in<Sndr, Env...>` constraint on the `completion_signatures_of_t<Sndr, Env...>` alias.

- Specify `get_completion_signatures(sndr, env)` to dispatch to `get_completion_signatures(sndr)` as a last resort, per suggestion from Lewis Baker.

- Add encouragement for implementors to use the completion signatures of the sender adaptor algorithms to propagate type errors.

- Add a design discussion about the decision to *infer* that types returned from `get_completion_signatures` represent errors if they are not specializations of `completion_signatures<>`.

R1:

- Change the specification of `transform_completion_signatures` to propagate types that are not specialization of the `completion_signatures<>` class template. This makes it easier to use an algorithm's completion signatures to communicate type errors from child senders.
- For the customization points `let_value`, `let_error`, and `let_stopped`, mandate that the callable's possible return types all satisfy `sender`.
- Change *Requires:* to *Mandates*: for algorithms that eagerly connect senders.

R0:

- Original revision

Improving early diagnostics

Problem Description

Type-checking a sender expression involves computing its completion signatures. In the general case, a sender's completion signatures may depend on the receiver's execution environment. For example, the sender:

```
read_env(get_stop_token)
```

... when connected to a receiver `rcvr` and started, will fetch the stop token from the receiver's environment and then pass it back to the receiver, as follows:

```
auto st = get_stop_token(get_env(rcvr));
set_value(move(rcvr), move(st));
```

Without an execution environment, the sender `read_env(get_stop_token)` doesn't know how it will complete.

The type of the environment is known rather late, when the sender is connected to a receiver. This is often far from where the sender expression was constructed. If there are type errors in a sender expression, those errors will be diagnosed far from where the error was made, which makes it harder to know the source of the problem.

It would be far preferable to issue diagnostics while *constructing* the sender rather than waiting until it is connected to a receiver.

Non-dependent senders

The majority of senders have completions that do not depend on the receiver's environment. Consider `just(42)` -- it will complete with the integer `42` no matter what receiver it is connected to. If a so-called "non-dependent" sender advertised itself as such, then sender algorithms could eagerly type-check the non-dependent senders they are passed, giving immediate feedback to the developer.

For example, this expression should be immediately rejected:

```
just(42) | then([](int* p) { return *p; })
```

The `then` algorithm can reject `just(42)` and the above lambda because the arguments don't match: an integer cannot be passed to a function expecting an `int*`. The `then` algorithm can do that type-checking only when it knows the

input sender is non-dependent. It couldn't, for example, do any type-checking if the input sender were `read_env(get_stop_token)` instead of `just(42)`.

And in fact, some senders *do* advertise themselves as non-dependent, although the sender algorithms in ([exec]) do not currently do anything with that extra information. A sender can declare its completions signatures with a nested type alias, as follows:

```
template <class T>
struct just_sender {
  T value;

  using completion_signatures =
    std::execution::completion_signatures<
      std::execution::set_value_t(T)
    >;

  // ...
};
```

Senders whose completions depend on the execution environment cannot declare their completion signatures this way. Instead, they must define a `get_completion_signatures` customization that takes the environment as an argument.

We can use this extra bit of information to define a `non_dependent_sender` concept as follows:

```
template <class Sndr>
concept non_dependent_sender =
  sender<Sndr> &&
  requires {
    typename remove_reference_t<Sndr>::completion_signatures;
  };
```

A sender algorithm can use this concept to conditionally dispatch to code that does eager type-checking.

Suggested Solution

The author suggests that this notion of non-dependent senders be given fuller treatment in `std::execution`. Conditionally defining the nested typedef in generic sender adaptors -- which may adapt either dependent or non-dependent senders -- is awkward and verbose. We suggest instead to support calling `get_completion_signatures` either with *or without* an execution environment. This makes it easier for authors of sender adaptors to preserve the "non-dependent" property of the senders it wraps.

We suggest that a similar change be made to the `completion_signatures_of_t` alias template. When instantiated with only a sender type, it should compute the non-dependent completion signatures, or be ill-formed.

Comparison Table

Consider the following code, which contains a type error:

```
auto work = just(42)
          | then([](int* p) { // <<< ERROR here
              //...
            });
```

The table below shows the result of compiling this code both before the proposed change and after:

Before	After
no error	`error: static_assert failed due to requirement '_is_completion_signatures< ustdex::ERROR<ustdex::WHERE (ustdex::IN_ALGORITHM, ustdex::then_t), ustdex ::WHAT (ustdex::FUNCTION_IS_NOT_CALLABLE), ustdex::WITH_FUNCTION ((lambda at hello.cpp:57:18)), ustdex::WITH_ARGUMENTS (int)>>' static_assert(_is_completion_signatures<_completions>); ^ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~`

This error was generated with with µstdex library and Clang-13.

Design Considerations

Why have two ways for non-dependent senders to publish their completion signatures?

The addition of support for a customization of `get_completion_signatures` that does not take an environment obviates the need to support the use of a nested `::completion_signatures` alias. In a class, this:

```
auto get_completion_signatures() ->
    std::execution::completion_signatures<
        std::execution::set_value_t(T)
    >;
```

... works just as well as this:

```
using completion_signatures =
    std::execution::completion_signatures<
        std::execution::set_value_t(T)
    >;
```

Without a doubt, we could simplify the design by dropping support for the latter. This paper suggests retaining it, though. For something like the `just_sender`, providing type metadata with an alias is more idiomatic and less surprising, in the author's opinion, than defining a function and putting the metadata in the return type. That is the case for keeping the `typename Sndr::completion_signatures` form.

The case for adding the `sndr.get_completion_signatures()` form is that it makes it simpler for sender adaptors such as `then_sender` to preserve the "non-dependent" property of the senders it adapts. For instance, one could define `then_sender` like:

```
template <class Sndr, class Fun>
struct then_sender {
    Sndr sndr_;
    Fun fun_;

    template <class... Env>
    auto get_completion_signatures(Env&&... env) const
      -> some-computed-type;

    //....
};
```

... and with this one member function support both dependent and non-dependent senders while preserving the "non-dependent-ness" of the adapted sender.

Proposed Wording

[Editorial note: This proposed wording is based on the current working draft. -- *end note]*

[Editorial note: Change [async.ops]/13 as follows: -- *end note]*

13. A completion signature is a function type that describes a completion operation. An asychronous operation has a finite set of possible completion signatures corresponding to the completion operations that the asynchronous operation potentially evaluates ([basic.def.odr]). For a completion function `set`, receiver `rcvr`, and pack of arguments `args`, let `c` be the completion operation `set(rcvr, args...)`, and let `F` be the function type `decltype(auto(set))(decltype((args))...)`. A completion signature `Sig` is associated with `c` if and only if `MATCHING-SIG(Sig, F)` is `true` ([exec.general]). Together, a sender type and an environment type `Env` determine the set of completion signatures of an asynchronous operation that results from connecting the sender with a receiver that has an environment of type `Env`. The type of the receiver does not affect an asychronous operation's completion signatures, only the type of the receiver's environment. A sender type whose completion signatures are knowable independent of an execution environment is known as a ***non-dependent sender***.

[Editorial note: Change [exec.syn] as follows: -- *end note]*

```
...
template<class Sndr, class... Env = env<>>
  concept sender_in = see below;
...

template<class Sndr, class... Env = env<>>
   requires sender_in<Sndr, Env...>
   using completion_signatures_of_t = call-result-t<get_completion_signatures_t, Sndr,
Env...>;
...

template<class Sndr, class... Env>
  using single-sender-value-type = see below;                    // exposition only

template<class Sndr, class... Env>
  concept single-sender = see below;                             // exposition only
...
```

1. The exposition-only type *`variant-or-empty`*`<Ts...>` is defined as follows ... as before

2. For types `Sndr` and pack `Env`, let `CS` be `completion_signatures_of_t<Sndr, Env...>`. Then *`single-sender-value-type`*`<Sndr, Env...>` is ill-formed if `CS` is ill-formed or if `sizeof...(Env) > 1` is `true`; otherwise, it is an alias for:

   - ~~`value_types_of_t<Sndr, Env`~~*`gather-signatures`*`<set_value_t, CS, decay_t, type_identity_t>` if that type is well-formed,
   - Otherwise, `void` if ~~`value_types_of_t<Sndr, Env`~~*`gather-signatures`*`<set_value_t, CS, tuple, variant>` is `variant<tuple<>>` or `variant<>`,
   - Otherwise, ~~`value_types_of_t<Sndr, Env`~~*`gather-signatures`*`<set_value_t, CS,` *`decayed-tuple`*`, type_identity_t>` if that type is well-formed,
   - Otherwise, *`single-sender-value-type`*`<Sndr, Env...>` is ill-formed.

3. The exposition-only concept *`single-sender`* is defined as follows:

```
namespace std::execution {
  template<class Sndr, class... Env>
```

```
    concept single-sender = sender_in<Sndr, Env...> &&
      requires {
        typename single-sender-value-type<Sndr, Env...>;
      };
  }
```

[Editorial note: Change [exec.snd.general] para 1 as follows: -- *end note]*

1. Subclauses [exec.factories] and [exec.adapt] define customizable algorithms that return senders. Each algorithm has a default implementation. Let `sndr` be the result of an invocation of such an algorithm or an object equal to the result ([concepts.equality]), and let `Sndr` be `decltype((sndr))`. Let `rcvr` be a receiver of type `Rcvr` with associated environment `env` of type `Env` such that `sender_to<Sndr, Rcvr>` is `true`. For the default implementation of the algorithm that produced `sndr`, connecting `sndr` to `rcvr` and starting the resulting operation state ([exec.async.ops]) necessarily results in the potential evaluation ([basic.def.odr]) of a set of completion operations whose first argument is a subexpression equal to `rcvr`. Let `Sigs` be a pack of completion signatures corresponding to this set of completion operations~~. Then~~ , and let `CS` be the type of the expression `get_completion_signatures(sndr, env)` . Then `CS` is a specialization of the class template `completion_signatures` ([exec.util.cmplsig]), the set of whose template arguments is `Sigs`. If none of the types in `Sigs` are dependent on the type `Env`, then the expression `get_completion_signatures(sndr)` is well-formed and its type is `CS`. If a user-provided implementation of the algorithm that produced `sndr` is selected instead of the default: *[Editorial note:* Reformatted into a list. -- *end note]*

    - Any completion signature that is in the set of types denoted by `completion_signatures_of_t<Sndr, Env>` and that is not part of `Sigs` shall correspond to error or stopped completion operations, unless otherwise specified.

    - If none of the types in `Sigs` are dependent on the type `Env`, then `completion_signatures_of_t<Sndr>` and `completion_signatures_of_t<Sndr, Env>` shall denote the same type.

[Editorial note: In [exec.snd.expos] para 24, change the definition of the exposition-only templates *`completion-signatures-for`* and *`basic-sender`* as follows: -- *end note]*

```
template<class Sndr, class... Env>
using completion-signatures-for = see below;                    // exposition only

template<class Tag, class Data, class... Child>
struct basic-sender : product-type<Tag, Data, Child...> {    // exposition only
  using sender_concept = sender_t;
  using indices-for = index_sequence_for<Child...>;       // exposition only

  decltype(auto) get_env() const noexcept {
    auto& [_, data, ...child] = *this;
    return impls-for<Tag>::get-attrs(data, child...);
  }

  template<decays-to<basic-sender> Self, receiver Rcvr>
  auto connect(this Self&& self, Rcvr rcvr) noexcept(see below)
    -> basic-operation<Self, Rcvr> {
    return {std::forward<Self>(self), std::move(rcvr)};
  }

  template<decays-to<basic-sender> Self, class... Env>
  auto get_completion_signatures(this Self&& self, Env&&... env) noexcept
    -> completion-signatures-for<Self, Env...> {
    return {};
  }
};
```

[Editorial note: Change [exec.snd.expos] para 39 as follows (this includes the proposed resolution of cplusplus/sender-receiver#307): -- *end note]*

39. For a subexpression `sndr` let `Sndr` be `decltype((sndr))`. Let `rcvr` be a receiver with an associated environment of type `Env` such that `sender_in<Sndr, Env>` is `true`. *`completion-signatures-for`*`<Sndr, Env>` denotes a specialization of `completion_signatures`, the set of whose template arguments correspond to the set of completion operations that are potentially evaluated as a result of starting ([exec.async.ops]) the operation state that results from connecting `sndr` and `rcvr`. When `sender_in<Sndr, Env>` is `false`, the type denoted by *`completion-signatures-for`*`<Sndr, Env>`, if any, is not a specialization of `completion_signatures`.

    Recommended practice: When `sender_in<Sndr, Env>` is `false`, implementations are encouraged to use the type denoted by *`completion-signatures-for`*`<Sndr, Env>` to communicate to users why.

39. Let `Sndr` be a (possibly `const`-qualified) specialization *`basic-sender`* or an lvalue reference of such, let `Rcvr` be the type of a receiver with an associated environment of type `Env`. If the type *`basic-operation`*`<Sndr, Rcvr>` is well-formed, let `op` be an lvalue subexpression of that type. Then *`completion-signatures-for`*`<Sndr, Env>` denotes a specialization of `completion_signatures`, the set of whose template arguments corresponds to the set of completion operations that are potentially evaluated ([basic.def.odr]) as a result of evaluating `op.start()`. Otherwise, *`completion-signatures-for`*`<Sndr, Env>` is ill-formed. If *`completion-signatures-for`*`<Sndr, Env>` is well-formed and its type is not dependent upon the type `Env`, *`completion-signatures-for`*`<Sndr>` is well-formed and denotes the same type; otherwise, *`completion-signatures-for`*`<Sndr>` is ill-formed.

[Editorial note: Change the `sender_in` concept in [exec.snd.concepts] para 1 as follows: -- *end note]*

```
template<class Sndr, class... Env ~~= env<>~~>
  concept sender_in =
    sender<Sndr> &&
    (sizeof...(Env) <= 1)
    (queryable<Env> &&...) &&
    requires (Sndr&& sndr, Env&&... env) {
      { get_completion_signatures(std::forward<Sndr>(sndr), std::forward<Env>(env)...) }
        -> valid-completion-signatures;
    };
```

[Editorial note: This subtly changes the meaning of `sender_in<Sndr>`. Before the change, it tests whether a type is a sender when used specifically with the environment `env<>`. After the change, it tests whether a type is a non-dependent sender. This is a stronger assertion to make about the type; it says that this type is a sender *regardless of the environment*. One can still get the old behavior with `sender_in<Sndr, env<>>`. -- *end note]*

[Editorial note: Change [exec.snd.concepts] para 4 as follows (so that the exposition-only *`sender-of`* concept tests for sender-ness with no environment as opposed to the empty environment, `env<>`): -- *end note]*

4. The exposition-only concepts *`sender-of`* and *`sender-in-of`* define the requirements for a sender type that completes with a given unique set of value result types.

```
namespace std::execution {
  template<class... As>
    using value-signature = set_value_t(As...);      // exposition only

  template<class Sndr, class Env, class... Values>
    concept sender-in-of =
      sender_in<Sndr, Env> &&
      MATCHING-SIG(                     // see [exec.general]
        set_value_t(Values...),
        value_types_of_t<Sndr, Env, value-signature, type_identity_t>);

  template<class Sndr, class... Values>
    concept sender-of = sender-in-of<Sndr, env<>, Values...>;

  template<class Sndr, class SetValue, class... Env>
    concept sender-in-of-impl =         // exposition only
      sender_in<Sndr, Env...> &&
      MATCHING-SIG(SetValue,                            // see [exec.general]
                   gather-signatures<set_value_t,       // see [exec.util.cmplsig]
```

```
                                        completion_signatures_of_t<Sndr, Env...>,
                                        value-signature,
                                        type_identity_t>);

  template<class Sndr, class Env, class... Values>
    concept sender-in-of =              // exposition only
      sender-in-of-impl<Sndr, set_value_t(Values...), Env>;

  template<class Sndr, class... Values>
    concept sender-of =                 // exposition only
      sender-in-of-impl<Sndr, set_value_t(Values...)>;

}
```

[Editorial note: Change [exec.awaitables] p 1-4 as follows: -- *end note]*

1. The sender concepts recognize awaitables as senders. For [exec], an *awaitable* is an expression that would be well-formed as the operand of a `co_await` expression within a given context.

2. For a subexpression `c`, let `GET-AWAITER(c, p)` be expression-equivalent to the series of transformations and conversions applied to `c` as the operand of an *await-expression* in a coroutine, resulting in lvalue `e` as described by [expr.await], where `p` is an lvalue referring to the coroutine's promise, which has type `Promise`.

   [*Note 1*: This includes the invocation of the promise type's `await_transform` member if any, the invocation of the `operator co_await` picked by overload resolution if any, and any necessary implicit conversions and materializations. -- *end note*]

   Let `GET-AWAITER(c)` be expression-equivalent to `GET-AWAITER(c, q)` where `q` is an lvalue of an unspecified empty class type *`none-such`* that lacks an `await_transform` member, and where `coroutine_handle<`*`none-such`*`>` behaves as `coroutine_handle<void>`.

3. Let *`is-awaitable`* be the following exposition-only concept:

```
template<class T>
concept await-suspend-result = see below;

template<class A, class... Promise>
concept is-awaiter = // exposition only
    requires (A& a, coroutine_handle<Promise...> h) {
        a.await_ready() ? 1 : 0;
        { a.await_suspend(h) } -> await-suspend-result;
        a.await_resume();
    };

template<class C, class... Promise>
concept is-awaitable =
    requires (C (*fc)() noexcept, Promise&... p) {
        { GET-AWAITER(fc(), p...) } -> is-awaiter<Promise...>;
    };
```

   `await-suspend-result``<T>` is `true` if and only if one of the following is `true`:

   - `T` is `void`, or
   - `T` is `bool`, or
   - `T` is a specialization of `coroutine_handle`.

4. For a subexpression `c` such that `decltype((c))` is type `C`, and an lvalue `p` of type `Promise`, *`await-result-type`*`<C, Promise>` denotes the type `decltype(`*`GET-AWAITER`*`(c, p).await_resume())` and *`await-result-type`*`<C>` denotes the type `decltype(`*`GET-AWAITER`*`(c).await_resume())`.

[Editorial note: Change [exec.getcomplsigs] as follows: -- *end note]*

1. `get_completion_signatures` is a customization point object. Let `sndr` be an expression such that `decltype((sndr))` is `Sndr`, and let `env` be ~~an expression such that `decltype((env))` is `Env`~~ a pack of zero or one expression. ~~Let~~If `sizeof...(env) == 0` is `true`, let `new_sndr` be `sndr`; otherwise, let `new_sndr` be the expression `transform_sender(decltype(`*`get-domain-late`*`(sndr, env...)){}, sndr, env...)`.~~, and let~~ Let `NewSndr` be `decltype((new_sndr))`. Then `get_completion_signatures(sndr, env...)` is expression-equivalent to `(void(sndr), void(env)..., CS())` except that `void(sndr)` and `void(env)...` are indeterminately sequenced, where `CS` is:

    1. `decltype(new_sndr.get_completion_signatures(env...))` if that type is well-formed,

    2. Otherwise, if `sizeof...(env) == 1` is `true`, then `decltype(new_sndr.get_completion_signatures())` if that expression is well-formed,

    3. Otherwise, `remove_cvref_t<NewSndr>::completion_signatures` if that type is well-formed,

    4. Otherwise, if *`is-awaitable`*`<NewSndr, `*`env-promise`*`<`~~`Env`~~`decltype((env))>...>` is `true`, then:

```
completion_signatures<
                          SET-VALUE-SIG(await-result-type<NewSndr, env-
promise<Envdecltype((env))>...>), // see [exec.snd.concepts]
      set_error_t(exception_ptr),
      set_stopped_t()>
```

    5. Otherwise, `CS` is ill-formed.

2. If `get_completion_signatures(sndr)` is well-formed and its type denotes a specialization of the `completion_signatures` class template, then `Sndr` is a non-dependent sender type ([async.ops]).

3. Given a type `Env`, if `completion_signatures_of_t<Sndr>` and `completion_signatures_of_t<Sndr, Env>` are both well-formed, they shall denote the same type.

4. Let `rcvr` be an rvalue whose type `Rcvr` *...as before*

[Editorial note: Change [exec.adapt.general] as follows: -- *end note]*

— When a parent sender is connected to a receiver `rcvr`, any receiver used to connect a child sender has an associated environment equal to *`FWD-ENV`*`(get_env(rcvr))`.

— An adaptor whose child senders are all non-dependent ([async.ops]) is itself non-dependent.

— These requirements apply to any function that is selected by the implementation of the sender adaptor.

— *Recommended practice*: Implementors are encouraged to use the completion signatures of the adaptors to communicate type errors to users and to propagate any such type errors from child senders.

[Editorial note: Change [exec.then] as follows: -- *end note]*

2. The names `then`, `upon_error`, and `upon_stopped` denote pipeable sender adaptor objects. For `then`, `upon_error`, and `upon_stopped`, let *`set-cpo`* be `set_value`, `set_error`, and `set_stopped` respectively. Let the expression *`then-cpo`* be one of `then`, `upon_error`, or `upon_stopped`. For subexpressions `sndr` and `f`, if `decltype((sndr))` does not satisfy `sender`, or `decltype((f))` does not satisfy *`movable-value`*, *`then-cpo`*`(sndr, f)` is ill-formed.

3. Otherwise, let *`invoke-result`* be an alias template such that *`invoke-result`*`<Ts...>` denotes the type `invoke_result_t<F, Ts...>` where `F` is the decayed type of `f`. The expression *`then-cpo`*`(sndr, f)`

mandates that either `sender_in<Sndr>` is `false` or the type *`gather-signatures`*`<decayed-typeof<set-cpo>, completion_signatures_of_t<Sndr>, invoke-result, type-list>` is well-formed.

4. ~~Otherwise, the~~The expression *`then-cpo`*`(sndr, f)` is expression-equivalent to:

```
transform_sender(get-domain-early(sndr), make-sender(then-cpo, f, sndr))
```

except that `sndr` is evaluated only once.

5. ~~For `then`, `upon_error`, and `upon_stopped`, let *`set-cpo`* be `set_value`, `set_error`, and `set_stopped` respectively.~~ The exposition-only class template *`impls-for`* ([exec.snd.general]) is specialized for *`then-cpo`* as follows: *...as before*

[Editorial note: Change [exec.let] by inserting a new paragraph between (3) and (4) as follows: -- *end note]*

4. Let *`invoke-result`* be an alias template such that *`invoke-result`*`<Ts...>` denotes the type `invoke_result_t<F, Ts...>` where `F` is the decayed type of `f`. The expression *`let-cpo`*`(sndr, f)` mandates that either `sender_in<Sndr>` is `false` or the type *`gather-signatures`*`<decayed-typeof<set-cpo>, completion_signatures_of_t<Sndr>, invoke-result, type-list>` is well-formed and that the types in the resulting type list all satisfy `sender`.

5. ~~Otherwise, the~~The expression *`let-cpo`*`(sndr, f)` is expression-equivalent to:

```
transform_sender(get-domain-early(sndr), make-sender(let-cpo, f, sndr))
```

except that `sndr` is evaluated only once.

[Editorial note: Change [exec.bulk] by inserting a new paragraph between (1) and (2) as follows: -- *end note]*

2. Let *`invoke-result`* be an alias template such that *`invoke-result`*`<Ts...>` denotes the type `invoke_result_t<F, Shape, Ts...>` where `F` is the decayed type of `f`. The expression `bulk(sndr, f)` mandates that either `sender_in<Sndr>` is `false` or the type *`gather-signatures`*`<set_value_t, completion_signatures_of_t<Sndr>, invoke-result, type-list>` is well-formed.

3. ~~Otherwise, the~~The expression `bulk(sndr, shape, f)` is expression-equivalent to:

```
transform_sender(get-domain-early(sndr), make-sender(bulk, product-type{shape f},
sndr))
```

except that `sndr` is evaluated only once.

[Editorial note: Change [exec.split] as follows: -- *end note]*

3. The name `split` denotes a pipeable sender adaptor object. For a subexpression `sndr`, let `Sndr` be `decltype((sndr))`. ~~If~~ The expression `split(sndr)` mandates that `sender_in<Sndr, split-env>` is `true` ~~`false`, `split(sndr)` is ill-formed~~.

4. ~~Otherwise, the~~The expression `split(sndr)` is expression-equivalent to:

```
transform_sender(get-domain-early(sndr), make-sender(split, {}, sndr))
```

except that `sndr` is evaluated only once.

[*Note 1*: The default implementation of `transform_sender` will have the effect of connecting the sender to a receiver. It will return a sender with a different tag type. -- *end note*]

[Editorial note: Change [exec.stopped.opt] as follows: -- *end note]*

2. The name `stopped_as_optional` denotes a pipeable sender adaptor object. For a subexpression `sndr`, let `Sndr` be `decltype((sndr))`. The expression `stopped_as_optional(sndr)` mandates that `!sender_in<Sndr> || single-sender<Sndr>` is `true`. The expression `stopped_as_optional(sndr)` is expression-equivalent to:

```
transform_sender(get-domain-early(sndr), make-sender(stopped_as_optional, {}, sndr))
```

except that `sndr` is only evaluated once.

3. Let `sndr` and `env` be subexpressions such that `Sndr` is `decltype((sndr))` and `Env` is `decltype((env))`. If *`sender-for`*`<Sndr, stopped_as_optional_t>` is `false`~~, or if the type *`single-sender-value-type`*`<Sndr, Env>` is ill-formed or `void`,~~ then the expression `stopped_as_optional.transform_sender(sndr, env)` is ill-formed; otherwise, that expression mandates that the type *`single-sender-value-type`*`<Sndr, Env>` is well-formed and not `void`, and ~~otherwise, it~~ is equivalent to: ... as before

[Editorial note: Change [exec.sync.wait] as follows: -- *end note]*

4. The name `this_thread::sync_wait` denotes a customization point object. For a subexpression `sndr`, let `Sndr` be `decltype((sndr))`. ~~If `sender_in<Sndr, sync-wait-env>` is `false`, the expression `this_thread::sync_wait(sndr)` is ill-formed. Otherwise, it~~ The expression `this_thread::sync_wait(sndr)` is expression-equivalent to the following, except that `sndr` is evaluated only once:

```
apply_sender(get-domain-early(sndr), sync_wait, sndr)
```

Mandates:

(4.1) — `sender_in<Sndr, sync-wait-env>` is `true`.

(4.2) — The type *`sync-wait-result-type`*`<Sndr>` is well-formed.

(4.3) — `same_as<decltype(e), sync-wait-result-type<Sndr>>` is `true`, where *`e`* is the `apply_sender` expression above.

...as before

[Editorial note: Change [exec.sync.wait.var] as follows: -- *end note]*

1. The name `this_thread::sync_wait_with_variant` denotes a customization point object. For a subexpression `sndr`, let `Sndr` be `decltype(into_variant(sndr))`. ~~If `sender_in<Sndr, sync-wait-env>` is `false`, the expression `this_thread::sync_wait(sndr)` is ill-formed. Otherwise, it~~ The expression `this_thread::sync_wait_with_variant(sndr)` is expression-equivalent to the following, except that `sndr` is evaluated only once:

```
apply_sender(get-domain-early(sndr), sync_wait_with_variant, sndr)
```

Mandates:

(1.1) — `sender_in<Sndr, sync-wait-env>` is `true`.

(1.2) — The type *`sync-wait-with-variant-result-type`*`<Sndr>` is well-formed.

(1.3) — `same_as<decltype(e), sync-wait-with-variant-result-type<Sndr>>` is `true`, where *`e`* is the `apply_sender` expression above.

2. ~~If *`callable`*`<sync_wait_t, Sndr>` is `false`, the expression `sync_wait_with_variant.apply_sender(sndr)` is ill-formed. Otherwise, it~~The expression `sync_wait_with_variant.apply_sender(sndr)` is equivalent to *...as before*

[Editorial note: Change [exec.run.loop.types] para 5 as follows: -- *end note]*

5. *run-loop-sender* is an exposition-only type that satisfies `sender`. ~~For any type `Env`,~~ `completion_signatures_of_t<` *run-loop-sender*~~`, Env`~~`>` is `completion_signatures<set_value_t(), set_error_t(exception_ptr), set_stopped_t()>`.

Acknowlegments

We owe our thanks to Ville Voutilainen who first noticed that most sender expressions could be type-checked eagerly but are not by P2300R8.